RecursiveCharacterTextSplitter

Fixed-Size Chunking with

chunk_size & chunk_overlap

A Practical Guide for LLM Engineers

Created: June 26, 2026 | 20:50

LangChain • Text Processing • RAG Pipelines

Table of Contents

1. Introduction to Text Chunking

Large Language Models (LLMs) have a fixed context window — a hard limit on how many tokens they can process in a single call. When working with documents that exceed this limit (articles, books, codebases, PDFs), you must **chunk** the text into smaller pieces before embedding or passing them to the model.

Text chunking is therefore a critical preprocessing step in:

- Retrieval-Augmented Generation (RAG) pipelines
- Semantic search over long documents
- Summarisation of large corpora
- Question-answering over private knowledge bases

> ■ **Note:** Poor chunking leads to broken context, duplicate retrieval results, and missed answers. Choosing the right strategy — and the right parameters — is as important as choosing your embedding model.

2. What is RecursiveCharacterTextSplitter?

RecursiveCharacterTextSplitter (RCTS) is LangChain's recommended general-purpose text splitter. It splits text using a *prioritised list of separators*, trying each one in order and recursively splitting pieces that are still too large.

The key idea: prefer semantic boundaries first (paragraph → line → word → character), falling back to smaller boundaries only when necessary. This keeps chunks as semantically coherent as possible while still respecting the hard **chunk_size** limit.

Figure 1 — RecursiveCharacterTextSplitter processing pipeline

3. Core Parameters: chunk_size & chunk_overlap

3.1 chunk_size

chunk_size sets the *maximum* number of characters (or tokens, depending on length_function) allowed in a single chunk. The splitter guarantees no produced chunk exceeds this value.

> ■ **Tip:** Start with chunk_size=512 for most RAG use-cases. If your embedding model supports 1024 tokens, you can safely go up to ~800 characters.

3.2 chunk_overlap

chunk_overlap controls how many characters from the *end* of the previous chunk are repeated at the *beginning* of the next chunk. Overlap ensures that sentences or ideas crossing a chunk boundary are not lost.

A typical rule of thumb: **chunk_overlap ≈ 10–20% of chunk_size**. Setting overlap too high wastes embedding budget; too low risks missing cross-boundary context.

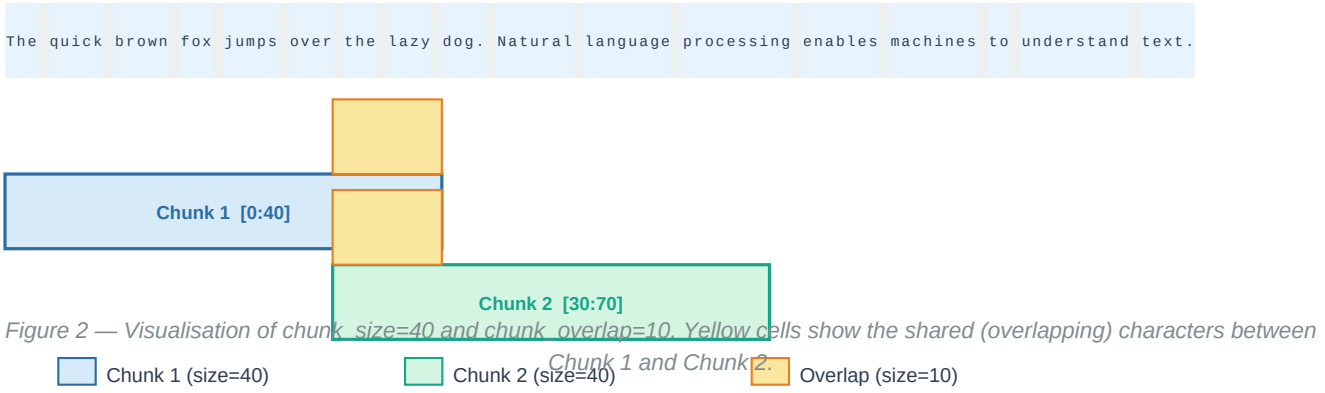


Figure 2 — Visualisation of chunk_size=40 and chunk_overlap=10. Yellow cells show the shared (overlapping) characters between Chunk 1 and Chunk 2.

3.3 All Parameters at a Glance

Parameter	Type	Default	Description
`chunk_size`	int	4000	Maximum number of characters per chunk
`chunk_overlap`	int	200	Characters shared between adjacent chunks
`separators`	List[str]	["\n\n","\n"," ",""]	Ordered list of split tokens
`length_function`	Callable	len	Function to measure chunk length
`is_separator_regex`	bool	False	Treat separators as regex patterns
`keep_separator`	bool	False	Include separator in the output chunk
`add_start_index`	bool	False	Attach char offset metadata to chunks
`strip_whitespace`	bool	True	Strip leading/trailing whitespace

Table 1 — Full parameter reference for RecursiveCharacterTextSplitter

4. How the Recursive Algorithm Works

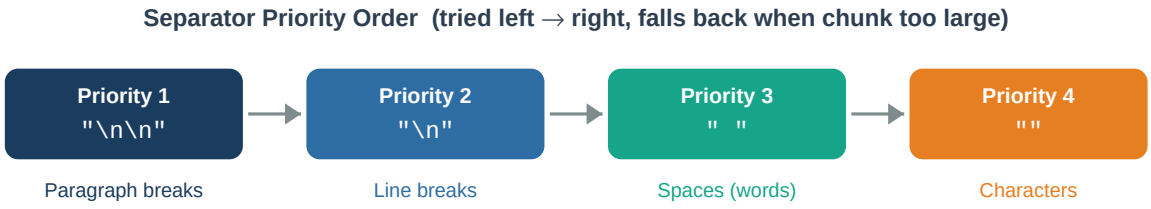
The algorithm follows a **divide-and-conquer** approach:

Step 1 — Choose separator	Pick the first (highest-priority) separator from the list.
Step 2 — Split	Split the input text on that separator, producing a list of sub-strings.
Step 3 — Measure	For each sub-string, check its length against chunk_size.
Step 4 — Recurse if needed	If a sub-string is still larger than chunk_size, recursively apply the algorithm with the next separator in the list.
Step 5 — Merge with overlap	Collect all sub-strings that fit within chunk_size and greedily merge them, prepending chunk_overlap characters from the previous chunk to each new one.
Step 6 — Output	Return the final list of chunks, each ≤ chunk_size characters.

■ **Warning:** If the *last* separator ('') is reached and a single character still exceeds chunk_size, that character is emitted as its own chunk. In practice this never happens with reasonable chunk_size values (≥ 50).

5. Default Separators & Hierarchy

By default, RCTS uses the separator list ["\n\n", "\n", " ", ""]. These map to natural text boundaries from coarsest to finest:



If a chunk still exceeds chunk_size after splitting, the next separator is tried recursively.

Figure 3 — Separator priority hierarchy (left = tried first)

Separator	Splits on	Preserves
"\n\n"	Double newline	Paragraph structure
"\n"	Single newline	Line structure

Separator	Splits on	Preserves
`" "`	Spaces	Word boundaries
`""`	Every character	Nothing (last resort)

Table 2 — Default separator list and what each one preserves

6. Installation & Basic Usage

6.1 Install LangChain

```
# Install core LangChain package
pip install langchain

# Or install only text-splitters (lighter dependency)
pip install langchain-text-splitters
```

6.2 Minimal Example

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """
Artificial intelligence is transforming industries worldwide.
Machine learning enables computers to learn from data.

Deep learning, a subset of machine learning, uses neural networks
with many layers to extract high-level features from raw input.
"""

splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=20,
)

chunks = splitter.split_text(text)
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1} ({len(chunk)} chars): {chunk[:60]}...")
```

Running the snippet above produces output similar to:

```
Chunk 1 (98 chars):  Artificial intelligence is transforming industries worldwide...
Chunk 2 (97 chars):  Machine learning enables computers to learn from data...
Chunk 3 (100 chars): Deep learning, a subset of machine learning, uses neural...
```

7. Python Code Examples

7.1 Splitting a File

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Read file content
with open("document.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64,
    length_function=len,          # character count
    add_start_index=True,         # track original positions
)

chunks = splitter.split_text(raw_text)
print(f"Total chunks: {len(chunks)}")
```

```
print(f"Avg chunk size: {sum(len(c) for c in chunks)//len(chunks)} chars")
```

7.2 Splitting LangChain Documents

When working with the LangChain Document abstraction (e.g., loaded from PDF, web pages, databases), use split_documents() instead of split_text():

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load document
loader = TextLoader("report.txt")
documents = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
)

# Split — metadata (source, page, etc.) is carried forward automatically
doc_chunks = splitter.split_documents(documents)

print(f"Original docs: {len(documents)}")
print(f"Chunks produced: {len(doc_chunks)}")
print("First chunk metadata:", doc_chunks[0].metadata)
```

7.3 Token-Based Chunking

LLM context limits are measured in **tokens**, not characters. One token ≈ 4 characters for English text, but this varies. Use the model's own tokeniser as length_function for precise control:

```
import tiktoken
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load the tokeniser for GPT-4 / text-embedding-ada-002
enc = tiktoken.get_encoding("cl100k_base")

def token_len(text: str) -> int:
    return len(enc.encode(text))

splitter = RecursiveCharacterTextSplitter(
    chunk_size=256,          # now means 256 TOKENS, not characters
    chunk_overlap=32,
    length_function=token_len,
)

chunks = splitter.split_text(some_long_text)
print(f"Max tokens in any chunk: {max(token_len(c) for c in chunks)}")
```

7.4 Inspecting Chunk Boundaries

Use add_start_index=True to attach the character offset of each chunk back to the original document. Useful for debugging and citation:

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=30,
    add_start_index=True,
)

docs = splitter.create_documents([long_text])

for doc in docs:
    start = doc.metadata["start_index"]
    end   = start + len(doc.page_content)
    print(f"[{start}:{end}]  {doc.page_content[:50]}...")
```

7.5 Using Regex Separators

Set is_separator_regex=True to use regular expressions as separators. This is powerful for splitting structured text like markdown or logs:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

markdown_splitter = RecursiveCharacterTextSplitter(
    separators=[
        r"\n#{1,6} ",   # any heading level (# through ######)
        r"\n---+",       # horizontal rules
        r"\n\n",        # paragraph
        r"\n",           # line
        r" ",             # word
        r"",              # char
    ],
    chunk_size=800,
```

```
    chunk_overlap=80,
    is_separator_regex=True,
)

chunks = markdown_splitter.split_text(markdown_content)
```

8. Advanced Usage & Customisation

8.1 Language-Specific Separators

RCTS provides built-in separator lists tuned for programming languages. Use from_language() to get the right separators for your code:

```
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter,
    Language,
)

# See all supported languages
print([e.value for e in Language])
# ['cpp', 'go', 'java', 'kotlin', 'js', 'ts', 'php', 'proto',
#  'python', 'rst', 'ruby', 'rust', 'scala', 'swift', 'markdown',
#  'latex', 'html', 'sol', 'csharp', 'cobol', 'c', 'lua', 'perl']

# Python-aware splitter
py_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=1000,
    chunk_overlap=100,
)

with open("my_module.py") as f:
    code = f.read()

code_chunks = py_splitter.split_text(code)
```

8.2 Custom Separator List

Supply your own separator list for domain-specific documents such as legal contracts, scientific papers, or CSV-like data:

```
legal_splitter = RecursiveCharacterTextSplitter(
    separators=[
        "\n\nARTICLE ",     # article breaks (highest priority)
        "\n\nSection ",     # section breaks
        "\n\n",             # paragraph
        "\n",               # line
        ". ",               # sentence
        " ",               # word
        "",                # character (fallback)
    ],
    chunk_size=1500,
    chunk_overlap=200,
)

clauses = legal_splitter.split_text(contract_text)
```

8.3 Batch Processing Many Files

```
import os
from pathlib import Path
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64,
)

all_chunks = []
for path in Path("docs/").glob("*.txt"):
    text = path.read_text(encoding="utf-8")
    for chunk in splitter.split_text(text):
        all_chunks.append({"source": path.name, "text": chunk})

print(f"Total chunks across all files: {len(all_chunks)}")
```

9. Performance & Chunk Size Trade-offs

Choosing the right chunk_size involves balancing three competing concerns:

Concern	Smaller chunks	Larger chunks
Retrieval precision	Higher — narrow, focused results	Lower — broader context returned
Context coherence	Lower — may cut mid-sentence	Higher — full paragraphs preserved
Embedding cost	Higher (more chunks to embed)	Lower (fewer chunks)
LLM token usage	Lower per query	Higher per query
Latency	Lower embedding time	Higher embedding time

Table 3 — Trade-off summary: smaller vs. larger chunks

9.1 Recommended Settings by Use Case

chunk_size	chunk_overlap	Chunks (est.)	Use Case
256	32	Many small	Dense retrieval, keyword search
512	64	Medium	Balanced RAG, semantic search
1024	128	Moderate	Contextual Q&A, summarisation
2048	256	Fewer large	Long-form reasoning, full-page context
4000	400	Very few	Whole-document tasks, big LLM windows

Table 4 — Suggested chunk_size / chunk_overlap combinations

■ **Tip:** Always evaluate chunking quality empirically. Run a small set of questions against your vector store and inspect which chunks are retrieved. Adjust chunk_size until retrieved chunks consistently contain the answer to your queries.

10. Comparison with Other Splitters

LangChain provides several text splitters. Here is how RCTS compares to the most commonly used alternatives:

Feature	CharacterTextSplitter	RecursiveCharacterTextSplitter
Split strategy	Single separator only	Multiple separators, tried in order
Default separators	User-defined string	["\n\n", "\n", " ", ""]
Recursion	No	Yes — falls back to next separator
Preserves structure	Sometimes	Yes — paragraphs > lines > words > chars
Best for	Simple/uniform text	General NLP, code, mixed content
Overlap support	Yes	Yes (chunk_overlap param)
LangChain class	CharacterTextSplitter	RecursiveCharacterTextSplitter

Table 5 — Splitter comparison matrix

■ **Note:** RecursiveCharacterTextSplitter is the recommended default for most use cases. Only switch to a specialised splitter (e.g., MarkdownHeaderTextSplitter, HTMLHeaderTextSplitter) when you need header-aware metadata or structural splitting.

11. Integration with RAG Pipelines

The most common production pattern: load documents → chunk → embed → store in a vector database → retrieve relevant chunks at query time.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_openai import ChatOpenAI
from langchain.chains import RetrievalQA

# 1. Load
loader = PyPDFLoader("company_handbook.pdf")
pages  = loader.load()

# 2. Chunk
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100,
    add_start_index=True,
)
chunks = splitter.split_documents(pages)

# 3. Embed & Store
embeddings  = OpenAIEmbeddings(model="text-embedding-ada-002")
```

```
vector_store = FAISS.from_documents(chunks, embeddings)

# 4. Retrieval QA
llm = ChatOpenAI(model="gpt-4o", temperature=0)
qa  = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_store.as_retriever(search_kwargs={"k": 4}),
)

answer = qa.invoke({"query": "What is the leave policy?"})
print(answer["result"])
```

12. Common Pitfalls & Best Practices

■ ***chunk_overlap* ≥ *chunk_size***	Setting overlap equal to or greater than chunk_size causes an infinite loop or ValueError. Always ensure chunk_overlap < chunk_size. A safe ratio is ≤ 25%.
■ ***Ignoring the length_function***	If your embedding model charges per token, using character-count (the default) gives inaccurate budget estimates. Switch to a tokeniser-based length_function when working with token-limited models.
■ ***Hardcoding chunk_size without testing***	chunk_size=1000 is a popular default but it may be too large for short-answer retrieval or too small for dense technical text. Always A/B test against your target queries.
■ ***Forgetting that RCTS strips whitespace***	By default strip_whitespace=True. If your downstream code relies on leading indentation (e.g., code snippets), set strip_whitespace=False.
■ ***Not preserving metadata***	When using split_text() instead of split_documents(), chunk source metadata (filename, page number, URL) is lost. Prefer split_documents() or manually attach metadata after splitting.
■ ***Very large files without streaming***	Loading a 100MB file entirely into memory before splitting can exhaust RAM. Use streaming loaders (e.g., UnstructuredFileLoader with mode='elements') to process large files lazily.

12.1 Quick Checklist

- ■ chunk_overlap is less than chunk_size (ideally 10–20%)
- ■ length_function matches your embedding model's token counter
- ■ add_start_index=True is set when you need citation support
- ■ split_documents() is used (not split_text()) to preserve metadata
- ■ Chunking output has been manually inspected on a sample document
- ■ chunk_size has been tested end-to-end against real retrieval queries
- ■ Language-specific separators are used for code files
- ■ strip_whitespace=False is set if preserving indentation matters

13. References

[1] LangChain Documentation — RecursiveCharacterTextSplitter
https://python.langchain.com/docs/how_to/recursive_text_splitter/

[2] LangChain API Reference — langchain_text_splitters https://api.python.langchain.com/en/latest/text_splitters/

[3] LangChain GitHub — text_splitter.py
https://github.com/langchain-ai/langchain/blob/master/libs/langchain/langchain/text_splitter.py

[4] OpenAI Cookbook — RAG Best Practices https://cookbook.openai.com/examples/how_to_build_a_rag_pipeline

[5] Anthropic — Contextual Retrieval https://www.anthropic.com/news/contextual-retrieval

[6] Pinecone — Chunking Strategies for LLM Applications https://www.pinecone.io/learn/chunking-strategies/

[7] tiktoken — OpenAI Tokeniser Library https://github.com/openai/tiktoken

[8] FAISS — Facebook AI Similarity Search https://github.com/facebookresearch/faiss

---

Document generated on June 26, 2026 at 20:50. Content is based on LangChain v0.2+ and Python 3.9+.